

Two-Matching Popular Winning Sets for Bipartite Graphs on at Most Six Vertices

Xiaojian Zeng

PBC School of Finance, Tsinghua University, 100190

xiaojianzeng@foxmail.com

September 7, 2026

Abstract

We prove an exact computer-assisted result for weighted bipartite matching instances with strict preferences and at most six vertices. For each total vertex order, there is a pair of matchings whose vertexwise best partners are at least as popular as every partial matching, simultaneously for all nonnegative real weights decreasing along that order. The certificates cover all 800 labeled graph domains for the three six-vertex bipartitions, 161,981 graph/preference combinations, and 3,101,711 profile/order cases. Integer vote-prefix inequalities certify each entire closed weight-order cone. Two separately implemented checkers verify the witnesses, by direct opponent enumeration and by a matching dynamic program. The two-matching bound follows for every finite bipartite graph whose connected components have at most six vertices. The unrestricted weighted marriage problem remains unresolved by this result.

1 Model and contribution

Let $G = (L \sqcup R, E)$ be a finite simple bipartite graph. Each vertex v has a strict preference order $\succ_v$ on its neighbors. Extend this order by putting the unmatched outcome $\perp$ below every neighbor. Write $\mathcal{M}(G)$ for all matchings of G , including the empty matching and nonmaximal matchings. For $N \in \mathcal{M}(G)$, let $N(v)$ be the partner of v , or $\perp$ when v is unmatched.

For a nonempty collection C of matchings, let $b_C(v)$ be the best of their partners at v . These vertexwise choices need not themselves form a matching. Define

$$c_v(C, N) = \begin{cases} 1, & b_C(v) \succ_v N(v), \\ -1, & N(v) \succ_v b_C(v), \\ 0, & b_C(v) = N(v). \end{cases}$$

For weights $w : V(G) \rightarrow \mathbb{R}_{\geq 0}$, the plurality margin is

$$\Delta_w(C, N) = \sum_{v \in V(G)} w(v) c_v(C, N). \quad (1)$$

Thus equality of partners is an abstention. A *popular winning set* is a collection C for which $\Delta_w(C, N) \geq 0$ for every $N \in \mathcal{M}(G)$. The popular dimension is the minimum size of such a collection. Allowing a pair A, B with $A = B$ expresses a bound of at most two.

Connor et al. [1] study this vertexwise-best comparison. Their Observation 3.2 and Theorem 3.3 give lower and upper bounds of two and three for the worst-case popular

dimension of weighted marriage instances with strict preferences. Their Section 5 asks for a bound of two, even when preference ties are allowed. The two-sided weighted problem is distinct from the one-sided matching model of Kavitha, Matuschke and Schmidt-Kraepelin [2], where matching voters occupy the agent side. In particular, their two-matching results do not directly establish the statement below with weighted voters on both sides.

For a total order $\pi = (v_1, \dots, v_s)$, put

$$K_\pi = \{w \in \mathbb{R}_{\geq 0}^{V(G)} : w(v_1) \geq \dots \geq w(v_s)\}.$$

Theorem 1. *Let G be a finite simple bipartite graph with at most six vertices, and give each vertex a strict order of its neighbors, with unmatched worst. For every total vertex order π , there exist $A, B \in \mathcal{M}(G)$, possibly equal, such that*

$$\Delta_w(\{A, B\}, N) \geq 0 \quad \text{for every } w \in K_\pi \text{ and every } N \in \mathcal{M}(G). \quad (2)$$

The same pair A, B works throughout the closed cone K_π .

The pair may depend on the graph, preferences and vertex order, but is chosen before the weight magnitudes. Equal and zero weights are included; preference ties are excluded. The theorem supplies an exhaustive initial vertex range beyond the known universal bound of three. Its contribution is the finite classification, including incomplete graphs and arbitrary real weights, together with explicit witnesses for every case. The summation identity used below is standard.

A bounded comparison of the two primary sources above, including the September 4, 2026 version of [1], found no equivalent six-vertex, cone-uniform statement as of September 7, 2026. This is a limited literature assessment, not an exhaustive priority claim. For broader adjacent context, we mention work on Pareto-efficient allocations on trees [4] and on proof-carrying finite graph reductions [3]. These references concern, respectively, preference-based allocation and certified graph arguments; neither is used as a theorem-level premise for our bound.

2 From real weights to integer prefixes

Lemma 2. *Fix G , its preferences, a total order $\pi = (v_1, \dots, v_s)$, a collection C , and an opponent N . Write $S_k(C, N) = \sum_{i=1}^k c_{v_i}(C, N)$. Then*

$$\Delta_w(C, N) \geq 0 \text{ for all } w \in K_\pi \iff S_k(C, N) \geq 0 \text{ for } 1 \leq k \leq s.$$

Proof. Set $w_i = w(v_i)$ and $w_{s+1} = 0$. Finite summation by parts gives

$$\sum_{i=1}^s w_i c_{v_i}(C, N) = \sum_{k=1}^s (w_k - w_{k+1}) S_k(C, N). \quad (3)$$

Indeed, the coefficient of $c_{v_i}(C, N)$ on the right is $\sum_{k=i}^s (w_k - w_{k+1}) = w_i$. Every coefficient $w_k - w_{k+1}$ is nonnegative for $w \in K_\pi$, proving sufficiency. Conversely, the weight vector which is one on $v_1, \dots, v_k$ and zero elsewhere belongs to K_π and has margin $S_k(C, N)$. This proves necessity. $\square$

Applying the lemma to every opponent shows that one fixed pair works throughout K_π exactly when all its opponent vote prefixes are nonnegative. All these prefixes are integers in $[-s, s]$. The reduction has no floating-point step and requires no sampling of weight magnitudes.

3 The finite certificate

3.1 Complete domain and encoding

On six vertices with both sides nonempty, exchanging the two sides leaves the bipartition sizes $(1, 5)$, $(2, 4)$ and $(3, 3)$. For side sizes (n, m) , where $n + m = 6$, label the left vertices $0, \dots, n - 1$ and the right vertices $n, \dots, 5$. Given any total order, relabel vertices within each side according to their positions in that order. The resulting order is an interleaving of the two ascending label lists. The graph and every preference list are relabeled at the same time. Since we enumerate every labeled edge subset and every strict profile, this operation loses no instance. There are $\binom{6}{n}$ interleavings.

For a graph H , there are $P(H) = \prod_{v \in V(H)} \deg_H(v)!$ strict profiles; an isolated vertex contributes $0! = 1$. Table 1 gives the complete counts. Graphs are labeled domains; isomorphic copies can occur more than once.

Sides	Graphs	Graph/profiles	Orders	Profile/order cases
$(1, 5)$	32	326	6	1,956
$(2, 4)$	256	26,669	15	400,035
$(3, 3)$	512	134,986	20	2,699,720
Total	800	161,981	—	3,101,711

Table 1: Exhaustive domain after relabeling within each bipartition side.

The accompanying supplement contains one binary file per row, with one unsigned byte per profile/order case. Each byte specifies a pair of maximal matchings, allowing repetition. A maximal matching is one to which no edge can be added. Restricting the stored witnesses to these matchings causes no logical difficulty: every decoded witness is a feasible matching and is checked directly. Opponents remain completely unrestricted.

The encoding is fixed as follows. Graph masks increase from zero to $2^{nm} - 1$, with bit $mi + j$ encoding the edge $(i, n + j)$. For each graph, orders are the lexicographic interleavings described above. For each order, preference profiles are the lexicographic product of the local lexicographic neighbor permutations, with vertex 5 varying fastest. All matchings are ordered by their left partner tuples, in lexicographic order, using -1 for unmatched and $0, \dots, m - 1$ for right partners. Maximal matchings retain this order. If their indices are $0, \dots, q - 1$, their pairs are ordered by (a, b) with $0 \leq a \leq b < q$, first by a and then by b . A byte is the zero-based index in this pair list.

Proposition 3 (Exact finite verification). *Every entry in the three certificate files decodes to a pair for which $S_k(\{A, B\}, N) \geq 0$ for every $1 \leq k \leq 6$ and every partial matching N in its graph, preference profile and order.*

Computer-assisted verification. The first C++ checker reconstructs the graph from its edge mask and enumerates all edge subsets, retaining exactly those having at most one incident selected edge at every vertex. This includes every partial matching exactly once. It sorts their partner tuples, identifies the maximal witnesses, reconstructs all local preference permutations, and decodes the certificate in the specified order.

For each matching, it obtains every partner’s rank by a direct search in the corresponding preference list, with unmatched appended last. For each certificate entry and each opponent, it compares the opponent rank with the smaller of the two committee ranks. In order π , these comparisons are added successively. The checker rejects a negative prefix, an invalid pair index, a truncated certificate or an extra byte. These steps test precisely the inequalities in the proposition. The complete successful execution checked 3,101,711 entries against 78,733,661

partial opponents and evaluated 472,401,966 integer prefix inequalities. Every prefix was nonnegative. The per-domain inequality counts are 58,716, 40,214,250 and 432,129,000, respectively.

The checkers and certificate files are supplied with a reproducible entry point in Appendix A. Prefix arithmetic is bounded by six in absolute value, and aggregate counts use 64-bit integers. This is an exhaustive finite verification of explicit witnesses; its trust basis includes the stated code, compiler and execution. $\square$

3.2 A second verification method

A separately written C++ checker verifies every entry by maximizing the opponent margin on each prefix. For a fixed committee and prefix $U = \{v_1, \dots, v_k\}$, let $t_v(p)$ be the vote for an opponent assigning partner p to v , compared with $b_C(v)$, and set $t_v(p) = 0$ when $v \notin U$. Thus $t_v(N(v)) = -c_v(C, N)$ on U . The empty opponent has score $B_U = \sum_v t_v(\perp)$. An edge $ab \in E$, with $a \in L$ and $b \in R$, has gain

$$g_{ab} = t_a(b) + t_b(a) - t_a(\perp) - t_b(\perp).$$

For every partial matching N , its opponent score is exactly

$$B_U + \sum_{ab \in N} g_{ab}. \quad (4)$$

The equality follows because disjoint edges replace the unmatched contributions at their endpoints.

A subset dynamic program processes left vertices in order. Its state $D_i(S)$ is the largest gain using the first i left vertices and exactly the right-vertex set S . Initially $D_0(\emptyset) = 0$ and other states are unreachable. Each transition either leaves the next left vertex unmatched or matches it to an unused neighbor b , adding g_{ab} and inserting b into S . Induction on i shows that these transitions enumerate all and only feasible partial matchings. Hence the maximum of (4) is $B_U + \max_S D_n(S)$.

All 18,610,266 optimized prefix problems, six per certificate entry, had nonpositive maxima. Direct enumeration cross-checked the dynamic program on the first profile of every graph/order and periodically thereafter, for 104,196 comparisons; every comparison agreed. The second checker also uses global partner labels, recursively enumerates witness matchings and profiles, and accesses certificate entries by offsets. These choices provide a separate implementation of the same finite statement. Both checkers rejected four deliberate corruptions: an out-of-range index, a legal pair with a negative first prefix, truncation, and an extra byte. This computational redundancy is not a formal proof-assistant verification.

4 Proof of the theorem and the component consequence

Proof of Theorem 1. First suppose G has six vertices and both bipartition sides are nonempty. Relabel within each side as above and, if necessary, exchange the sides. Proposition 3 supplies a pair for the resulting graph, strict profile and interleaving. Its prefix inequalities, together with Lemma 2, prove (2) on the entire closed cone. Transporting the pair back through the relabeling preserves all comparisons and the specified weight order.

An edgeless graph, including the graph with no vertices, is handled by $A = B = \emptyset$. For any other graph on fewer than six vertices, add isolated vertices to reach six and put them last in the total order. The padded graph has two nonempty sides, and after a side exchange has one of the three listed side sizes. Choose its pair by the preceding case. For every original $w \in K_\pi$, extending w by zeros gives an allowed weight vector for the padded

order. Every added vertex abstains, and all padded matchings restrict to matchings of the original graph. The restricted pair therefore works for every original w and every original opponent, as required. $\square$

Corollary 4. *Every finite strict weighted bipartite matching instance whose connected components have at most six vertices has popular dimension at most two. Consequently, a counterexample to the unrestricted two-matching assertion must contain a connected component with at least seven vertices.*

Proof. Order the vertices by nonincreasing weight, breaking weight ties arbitrarily. Restrict this order to each component H and use Theorem 1 to choose A_H, B_H . The unions $A = \bigcup_H A_H$ and $B = \bigcup_H B_H$ are matchings. Any global opponent N restricts to an opponent N_H in each component. As partners stay within components,

$$\Delta_w(\{A, B\}, N) = \sum_H \Delta_{w|_H}(\{A_H, B_H\}, N_H) \geq 0.$$

The counterexample-size statement is the contrapositive. $\square$

The result leaves the unrestricted original assertion unresolved. In particular, it makes no claim for connected graphs with seven or more vertices or for preference ties. The certificates provide a finite benchmark and exclude all instances assembled from components of size at most six. No general cone-uniform conjecture is inferred from this classification.

A Reproduction and assistance disclosure

The supplement contains `subgraphs15-certificate.bin`, `subgraphs24-certificate.bin` and `subgraphs33-certificate.bin`, of lengths 1,956, 400,035 and 2,699,720 bytes. It also contains the direct checker `verify_bipartite6.cpp`, the dynamic-programming checker `referee_verify.cpp`, a Python driver, and a file manifest. From the supplement directory, run

```
python3 verify.py
```

The driver uses Python’s standard library, compiles the two C++17 programs with an available Clang or GNU C++ compiler, and checks all three certificates with both programs. It verifies the expected coverage counts and runs the four corruption checks. Generated executables and logs go into a separate build directory. No certificate search or external solver is required for reproduction. The accompanying README specifies compiler selection and output files; the manifest binds the distributed data and checker sources.

The original witness generation used Python and NumPy with exact small integer signs and prefix sums. The final certificate verification uses only bounded integer comparisons and arithmetic. Exploratory weight searches and solver runs are not premises of the proof. The separate verification pass used C++17 with undefined-behavior sanitization; its domain and operation counts are reported in the text.

AI systems assisted with mathematical exploration, code generation, certificate construction, a separate mathematical review pass, and manuscript preparation. The separate review included the second checker and complete certificate replay. Separate AI-assisted passes are not statistically independent evidence or human peer review. No part of the argument has been formalized in a proof assistant.

References

- [1] Frank Connor, Louis-Roy Langevin, Ndiamé Ndiaye, Agnès Totschnig, Rohit Vasishta, and Adrian Vetta. The popular dimension of matchings, 2026. arXiv:2509.25150, version 3, September 4. <https://arxiv.org/abs/2509.25150v3>.
- [2] Telikepalli Kavitha, Jannik Matuschke, and Ulrike Schmidt-Kraepelin. Condorcet dimension and Pareto optimality for matchings and beyond, 2026. arXiv:2602.16289, version 1, February 18. <https://arxiv.org/abs/2602.16289v1>.
- [3] Zijian Zeng. Proof-carrying reductions for 4-regular girth-nine graphs of excess four. SSRN working paper 7374858, 2026. <https://doi.org/10.2139/ssrn.7374858>.
- [4] Zijian Zeng. Reachable globally Pareto-efficient allocations on trees are NP-complete. Preprints, 2026. <https://doi.org/10.20944/preprints202608.2133.v1>.